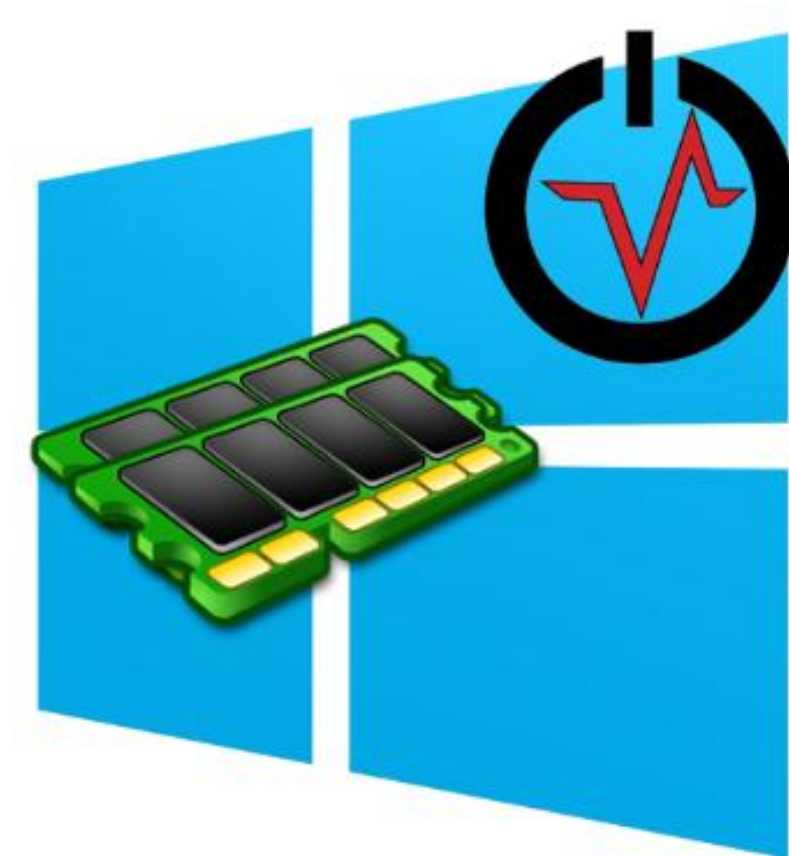


Análisis Forense Informático

Unidad 5. Análisis de memoria en Sistemas Windows



Índice

1	Introducción y contextualización.....	2
2	Análisis de memoria.....	2
2.1	Evidencias clave de memoria	3
2.2	Herramientas para la adquisición de memoria RAM	4
2.3	Análisis técnico de memoria	9
2.3.1	Volatility	10
2.3.1.1	Profiles y tablas de símbolos	12
2.3.1.2	Versiones de Volatility.....	12
2.3.1.3	Instalación.....	14
2.3.1.4	Funcionalidades de volatility 2.....	22
3	Listado de herramientas útiles.....	23
4	Referencias.....	23

1 Introducción y contextualización.

En el ámbito del análisis forense informático, el estudio de la **memoria volátil** y de los **sistemas de archivos** constituye una de las fases más críticas para la obtención de evidencia digital fiable.

El **análisis de un volcado de memoria** (memory dump) permite identificar procesos activos, conexiones de red, contraseñas en memoria y otros artefactos efímeros que pueden desaparecer tras el apagado del sistema.

En esta unidad se abordarán las técnicas y herramientas necesarias para realizar el análisis forense de un **volcado de memoria**, interpretando la información obtenida y comprendiendo su relevancia en el contexto de una investigación digital.

El objetivo es ser capaces de identificar, preservar y analizar estos tipos de evidencias, aplicando metodologías rigurosas que garanticen la validez y trazabilidad de los hallazgos.

2 Análisis de memoria.

El **análisis de memoria** en sistemas informáticos es una de las disciplinas más relevantes dentro del campo de la informática forense, ya que permite obtener **información crítica** directamente desde la **memoria volátil (RAM)** de un sistema en ejecución.

A diferencia del análisis de disco, que se centra en datos almacenados de forma persistente, el estudio de la memoria ofrece una instantánea del estado operativo del sistema en un momento determinado, revelando **evidencias que pueden desaparecer** al apagar o reiniciar el equipo.

A través del análisis de un **volcado de memoria**, es posible identificar procesos activos, conexiones de red, módulos cargados, sesiones de usuario, contraseñas temporales e incluso rastros de actividades maliciosas que no dejan huella en el sistema de archivos.

Este tipo de examen resulta especialmente valioso en investigaciones relacionadas con malware, intrusiones, robo de información o uso indebido de sistemas corporativos.

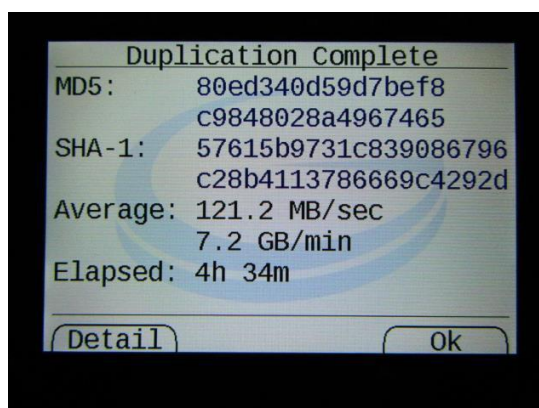
El objetivo de este tema es comprender la importancia del análisis de memoria dentro del proceso forense, conocer las principales herramientas utilizadas como **Volatility** y aprender a interpretar los artefactos obtenidos para reconstruir el comportamiento del sistema y las acciones del usuario.

2.1 Evidencias clave de memoria

Como ya sabemos, los análisis forenses pueden llevarse a cabo de dos formas (no excluyentes)

- Estática (en muerto) se analiza una copia forense
 - Solo se puede analizar lo persistente (sistema de ficheros, registro, logs de eventos, etc)
- Dinámica (en vivo) se analiza el dispositivo mientras está encendido
 - Se puede analizar tanto lo persistente como lo que hay en memoria

Nota: Aunque de ambas formas se puede obtener información interesante, en este apartado nos vamos a centrar principalmente en el análisis dinámico.



La información volátil disponible en la memoria del sistema puede contener evidencias clave para la resolución de un caso.

- Información del dispositivo y versión del sistema
- Fecha y hora del sistema
- Procesos y servicios en ejecución
- Conexiones de red abiertas y procesos asociados
- Usuarios conectados (y método de conexión)
- Archivos abiertos
- Variables de entorno
- Drivers instalados y en ejecución
- Tabla de enrutamiento
- ...

2.2 Herramientas para la adquisición de memoria RAM

Existen muchas herramientas destinadas a capturar la RAM del sistema Windows y volcarla en un archivo:

WinPmem

Utilizaremos esta herramienta para realizar la práctica, ya que es de las más útiles, más actualizadas y tiene su versión para Windows y aunque estamos basando el análisis en sistemas Windows, WinPmem también aporta una versión para extraer un volcado de memoria en sistemas basados en GNU/Linux. Por ello nos extenderemos un poco más que las demás, para entrar en una mayor profundidad.

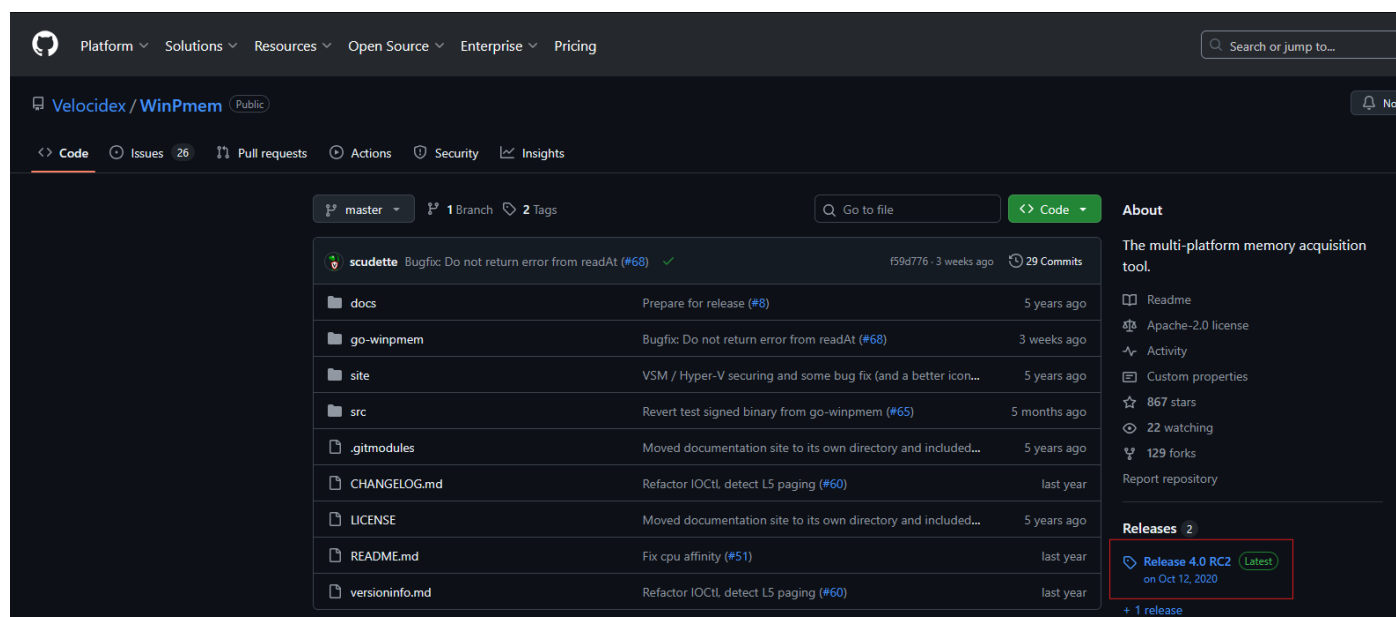
Podemos ver un ejemplo de adquisición en <https://winpmem.velocidex.com/docs/memory/>

A continuación, veremos como descargar la herramienta y como usarla para realizar un volcado de memoria.

Descarga y uso de WinPmem

En primer lugar, accederemos al repositorio oficial de la aplicación, para descargar el ejecutable, en nuestro caso para Windows.

<https://github.com/Velocidex/WinPmem>



The screenshot displays the GitHub repository for Velocidex/WinPmem. The repository is public and has 26 issues, 1 pull request, 29 commits, and 129 forks. The 'Releases' section on the right shows the latest release, 'Release 4.0 RC2', dated Oct 12, 2020, with a 'Latest' badge. The repository structure includes folders like docs, go-winpmem, site, and src, and files like CHANGELOG.md, LICENSE, README.md, and versioninfo.md.

Nos descargaremos el ejecutable correspondiente

▼

Assets

6

go-winpmem_amd64_1.0-rc1_signed.exe

4.75 MB

Jul 12, 2024

go-winpmem_amd64_1.0-rc2_signed.exe

sha256:86691bb4af2c17dd9ec4834c04a99...

5.01 MB

Sep 3

winpmem_mini_x64_rc2.exe

515 KB

Oct 13, 2020

winpmem_mini_x86.exe

212 KB

Oct 12, 2020

Source code (zip)

Oct 12, 2020

Source code (tar.gz)

Oct 12, 2020

👍

25

👏

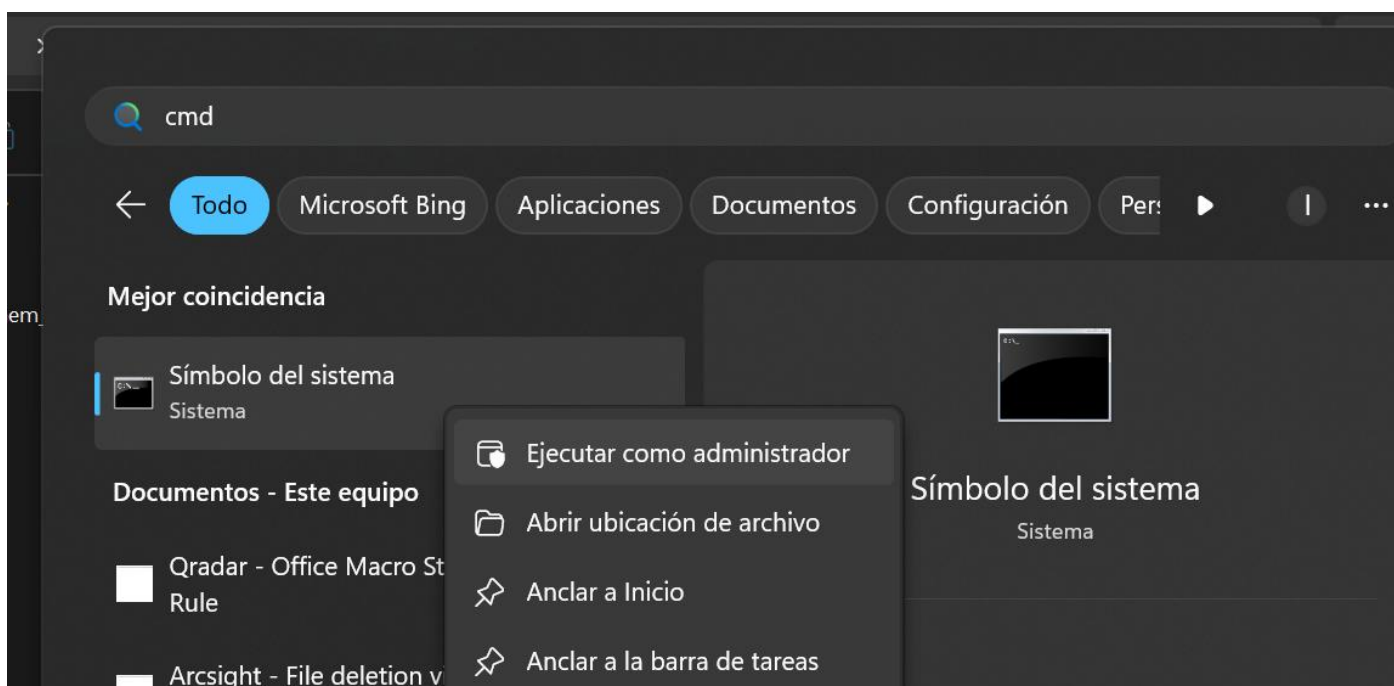
1

❤️

2

27 people reacted

Una vez tenemos el ejecutable iniciamos una consola de línea de comandos (con permisos de administrador)



En la consola con permisos de administración nos dirigimos a la carpeta donde tenemos nuestro ejecutable de winpmem

```

Administrador: Símbolo del sistema

C:\Users\ismael.gil\Downloads\Adquisición de memoria>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 143A-5AA9

Directorio de C:\Users\ismael.gil\Downloads\Adquisición de memoria

04/11/2025  14:30    <DIR>          .
04/11/2025  14:30    <DIR>          ..
04/11/2025  14:29             527.640 winpmem_mini_x64_rc2.exe
               1 archivos             527.640 bytes
               2 dirs  22.012.104.704 bytes libres

C:\Users\ismael.gil\Downloads\Adquisición de memoria>

```

Nota: Muy importante, en un entorno real no realizar el volcado de memoria en el mismo equipo, ya que ejecutar el volcado de memoria en el mismo equipo puede afectar al estado de la RAM y a la evidencia.

Una vez en la carpeta adecuada, ejecutamos la app y realizamos el volcado de la siguiente manera:

```
C:\Users\ismael.gil\Downloads\Adquisición de memoria>winpmem_mini_x64_rc2.exe Win1120251104.raw
WinPmem64
Extracting driver to C:\Users\ismael.gil\AppData\Local\Temp\pme9BE5.tmp
Driver Unloaded.
Loaded Driver C:\Users\ismael.gil\AppData\Local\Temp\pme9BE5.tmp.
Deleting C:\Users\ismael.gil\AppData\Local\Temp\pme9BE5.tmp
The system time is: 13:40:26
Will generate a RAW image
- buffer_size_: 0x1000
CR3: 0x00001AE000
4 memory ranges:
Start 0x00001000 - Length 0x0009E000
Start 0x00100000 - Length 0x60629000
Start 0x64FFF000 - Length 0x00001000
Start 0x100000000 - Length 0x38F800000
max_physical_memory_ 0x48f800000
Acquisition mode PTE Remapping
Padding from 0x00000000 to 0x00001000
pad
- length: 0x1000

00% 0x00000000 .
copy_memory
- start: 0x1000
- end: 0x9f000

00% 0x00001000 .
Padding from 0x0009F000 to 0x00100000
```

Se aconseja un nombre descriptivo, en este caso el nombre es Win11+timestamp del día en que se realiza el volcado.

Una vez, terminada la operación, tendremos nuestro volcado de memoria en la misma carpeta que tenemos el ejecutable

```
C:\> Administrador: Símbolo del sistema

39% 0x1C8000000 .....
43% 0x1FA000000 .....
47% 0x22C000000 .....
51% 0x25E000000 .....
56% 0x290000000 .....
60% 0x2C2000000 .....
64% 0x2F4000000 .....
69% 0x326000000 .....
73% 0x358000000 .....
77% 0x38A000000 .....
81% 0x3BC000000 .....
86% 0x3EE000000 .....
90% 0x420000000 .....
94% 0x452000000 .....
99% 0x484000000 ...x.x.....
The system time is: 13:41:13
Driver Unloaded.

C:\Users\ismael.gil\Downloads\Adquisición de memoria>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 143A-5AA9

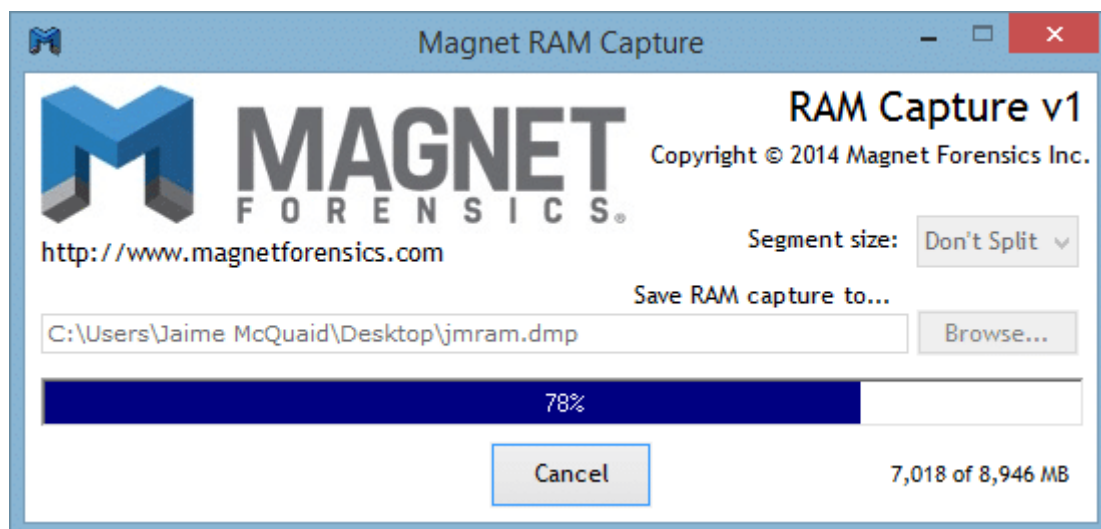
Directorio de C:\Users\ismael.gil\Downloads\Adquisición de memoria

04/11/2025  14:40    <DIR>          .
04/11/2025  14:30    <DIR>          ..
04/11/2025  14:41    19.587.399.680 Win1120251104.raw
04/11/2025  14:29           527.640 winpmem_mini_x64_rc2.exe
                2 archivos 19.587.927.320 bytes
```

Nota: Debemos tener en cuenta que la herramienta adquiere toda la memoria RAM (aunque no toda esté en uso). Por tanto, debemos prever guardar la adquisición en un lugar con espacio suficiente

Magnet RAM Capture

<https://www.magnetforensics.com/blog/acquiring-memory-with-magnet-ram-capture/>



Mandiant Memoryze

<https://www.sans.org/blog/digital-forensics-how-to-memory-analysis-with-mandiant-memoryze>

DumplIT

<https://www.magnetforensics.com/resources/magnet-dumpit-for-windows/>

FTK Imager

<https://www.exterro.com/ftk-product-downloads/ftk-imager-4-7-3-81>

LiME

<https://github.com/504ensicsLabs/LiME>

2.3 Análisis técnico de memoria

Una vez tenemos el volcado de memoria, debemos proceder a su análisis.

En la memoria RAM podemos encontrar **artefactos**, como:

- Procesos y servicios en ejecución
- Versiones desempaquetadas/descifradas de programas protegidos
- Información del sistema (p. ej. el tiempo transcurrido desde el último reinicio)
- Información sobre los usuarios conectados
- Información del registro
- Conexiones de red abiertas y la caché ARP
- Restos de chats, comunicaciones en redes sociales, juegos, etc.
- Actividades recientes de navegación web, incluyendo el modo incógnito de IE y modos similares orientados a la privacidad en otros navegadores web
- Comunicaciones recientes a través de sistemas de correo web
- Información de servicios en la nube
- Claves de descifrado para los volúmenes cifrados montados en el momento de la captura
- Imágenes visualizadas recientemente
- Software malicioso / Troyanos en ejecución

No obstante, el **análisis de la memoria RAM**, también tiene sus **limitaciones**:

- Muchos tipos de artefactos almacenados en la memoria volátil del ordenador son efímeros: ¡ahora están ahí y un minuto después ya no!
- La información sobre los procesos en ejecución permanecerá en memoria hasta que dichos procesos finalicen.
- Los restos de los chats recientes, comunicaciones y otras actividades del usuario pueden sobrescribirse con otro contenido en el momento en que el sistema operativo necesite una página de memoria libre.

2.3.1 Volatility



En esta unidad nos vamos a centrar en Volatility como herramienta para el análisis de memoria RAM. Esta herramienta nos permite:

- **Análisis profundo de la memoria RAM**
 - Permite examinar el contenido completo de la memoria volátil (RAM) de un sistema, ofreciendo una visión detallada del estado del equipo en el momento del volcado.
- **Recuperación de evidencias efímeras**
 - Facilita la extracción de datos que no se almacenan en disco, como procesos activos, conexiones de red, contraseñas en memoria, comandos ejecutados o sesiones de usuario.
- **Detección de actividad maliciosa**
 - Ayuda a identificar malware, rootkits, inyecciones de código y otros comportamientos sospechosos que pueden ocultarse del sistema operativo.
- **Compatible con múltiples sistemas operativos**
 - Soporta análisis de volcados de memoria de Windows, Linux, macOS y Android, lo que lo hace versátil para investigaciones en distintos entornos.

- **Amplia biblioteca de plugins**
 - Dispone de decenas de plugins especializados para extraer información específica (procesos, drivers, conexiones, historial, etc.), lo que permite adaptar el análisis a cada caso.
- **Herramienta gratuita y de código abierto**
 - Al ser open source, no requiere licencias comerciales y puede auditarse, modificarse y personalizarse según las necesidades del analista.
- **Compatibilidad con formatos estándar de volcado**
 - Acepta imágenes de memoria creadas con diversas herramientas (DumpIt, WinPmem, Magnet RAM Capture, FTK Imager, entre otras).
- **Formatos de entra permitidos**
 - Volcados sin procesar (raw)
 - Volcados de caída del sistema
 - Archivos de hibernación
 - Estados guardados en equipos virtuales como VmWare o VirtualBox
 - Volcados de LiME
 - Volcados EWF (Expert Witness Format)
 - Memoria física extraída directamente de una interfaz Firewire
- **Integración con otras herramientas forenses**
 - Puede combinarse con frameworks y scripts complementarios como Volatility Workbench, VolUtility o Autopsy, potenciando la automatización del análisis.
- **Amplia comunidad y documentación**
 - Al ser una de las herramientas más utilizadas en DFIR (Digital Forensics and Incident Response), cuenta con foros, guías, cursos y documentación extensa.
- **No altera la evidencia original**
 - Realiza el análisis sobre una copia del volcado de memoria, manteniendo la integridad de la evidencia digital y garantizando su validez en procesos legales.

2.3.1.1 Profiles y tablas de símbolos

Volatility utiliza información de depuración del kernel para localizar y obtener los objetos en memoria.

- Básicamente, el **profile** y las **tablas de símbolos** indican cómo se colocan los artefactos en la memoria RAM.
- Esta disposición de los artefactos depende de la distribución y del kernel que se utilice, tanto en Windows como en Linux.
- Esta información se guarda en **profiles** (Volatility 2) y en **tablas de símbolos** (Volatility 3).
- Se pueden encontrar los **profiles** en el GitHub de Volatility.
- Existen repositorios de **tablas de símbolos**, como por ejemplo [Volatility3 symbols \(2024\)](#).

2.3.1.2 Versiones de Volatility

Volatility v2

- Utiliza Python 2
- Utiliza "profiles" del sistema operativo
- Algunos análisis de imágenes de sistemas operativos antiguos solo se pueden hacer con la versión 2
- Más estable que la versión 3, pero hay sistemas actuales que ya no la soportan
- Necesario indicar el profile para el análisis
- Si no existe un profile adecuado se puede crear de manera manual.
 - <https://beguier.eu/nicolas/articles/security-tips-3-volatility-linux-profiles.html>
- Ejemplos de usos del plugin pslint, para obtener los procesos de una imagen

```
volatility -f MEMORY_FILE.raw --profile=WinXPSP2x86 pslint
```

```
volatility -f imatge-ubuntu-18.04-5.4.0-84-generic.lime --  
profile=LinuxUbuntu-18_04-5_4_0-84-genericx64 linux_pslint
```

Volatility v3

- Utiliza Python 3
- Actualmente es la versión que está en mantenimiento
- Más rápida y automática que la versión 2
- No utiliza “profiles”, ya que tiene una biblioteca extensa de tablas de símbolos y se pueden generar tablas de símbolos para la mayoría de las imágenes de memoria de Windows, basadas en la propia imagen.
- No es necesario indicarle la tabla de símbolos, la utiliza automáticamente
- Se pueden crear tablas de símbolos manualmente al igual que los perfiles de la versión 2, <https://beguier.eu/nicolas/articles/security-tips-3-volatility-linux-profiles.html>
- Ejemplo de usos del plugin pslist, para obtener los procesos de una imagen

```
python3 vol.py -f samples/cridex.vmem windows.pslist
```

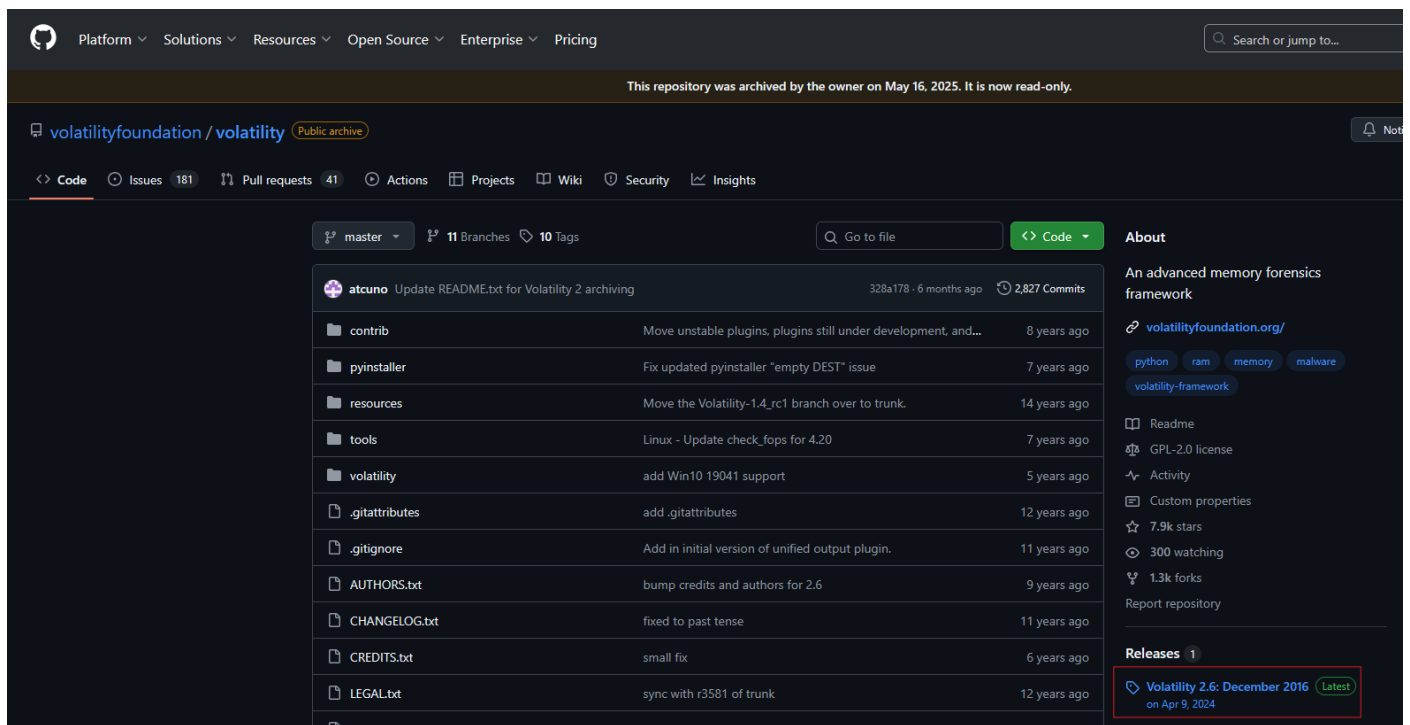
```
python3 vol.py -f imatge-ubuntu-18.04-5.4.0-84-generic.lime  
linux.pslist
```

Enlace de interés: [Equivalencia de comandos Vol2 y Vol3](#)

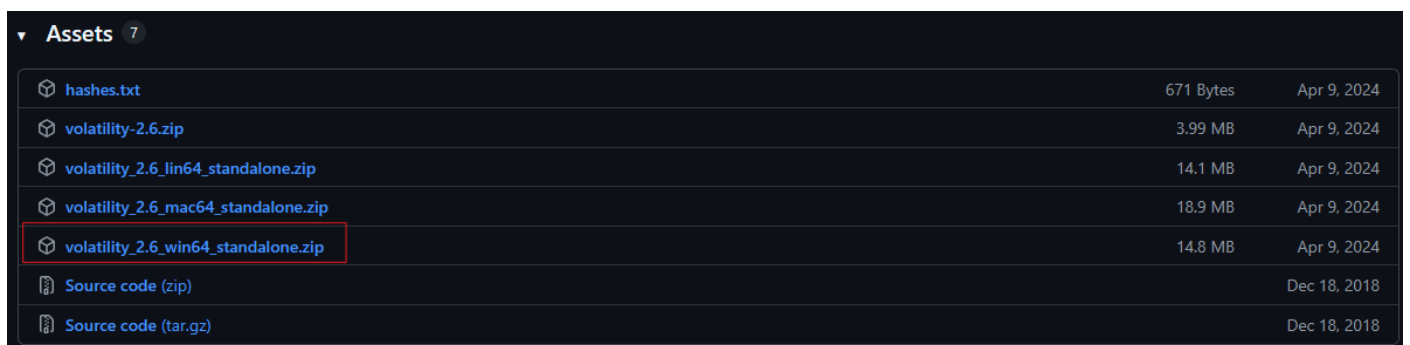
2.3.1.3 Instalación

Volatility V2 (Windows)

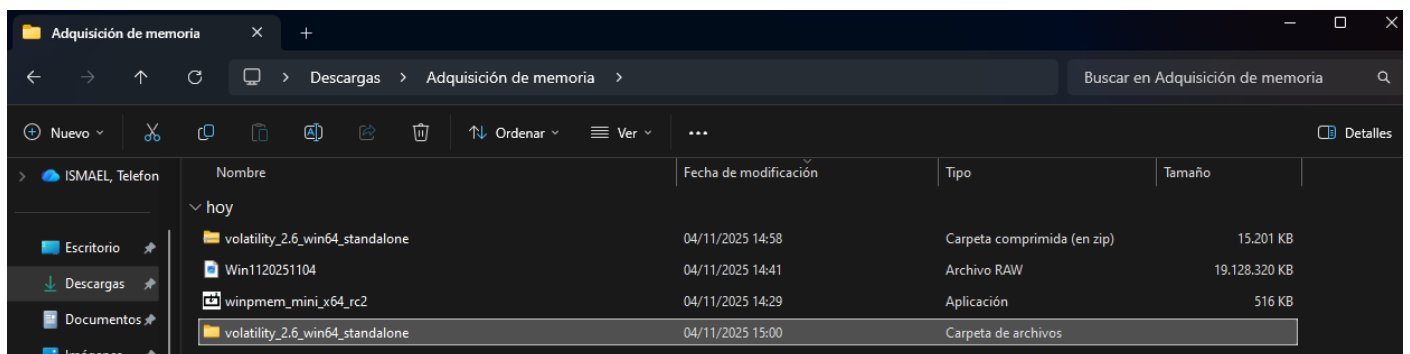
Para instalar Volatility en su versión 2, la más utilizada actualmente, tendremos que ir al repositorio oficial.



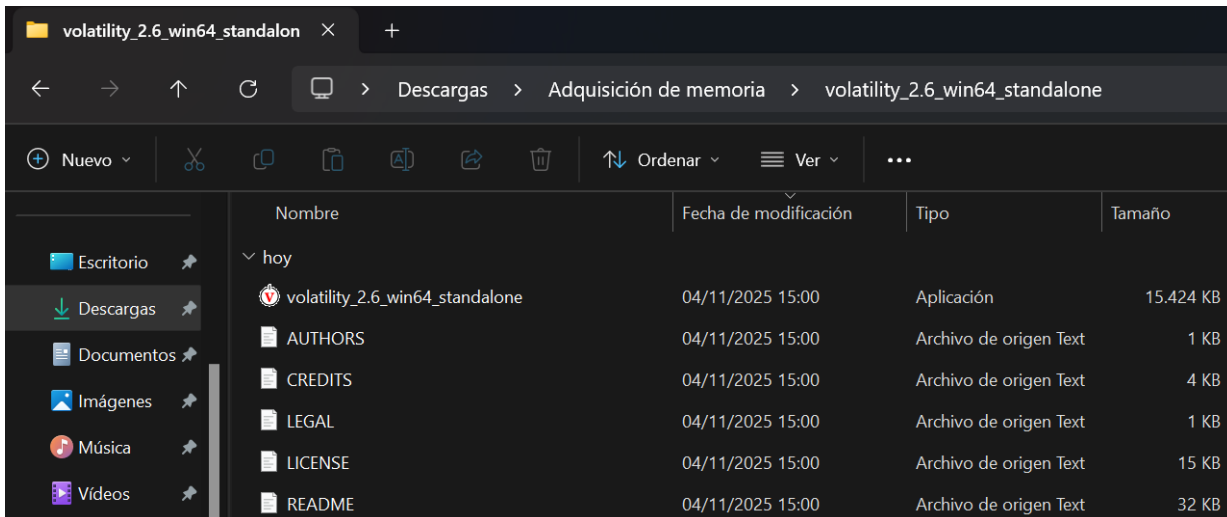
Nos descargaremos su última versión



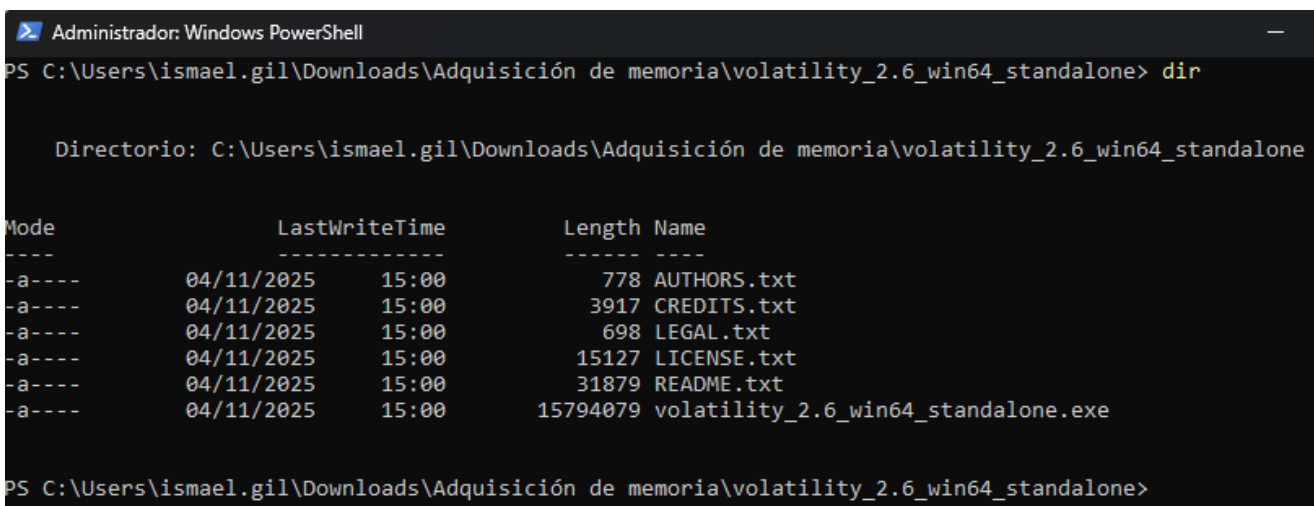
Una vez descargada, descomprimos el archivo descargado



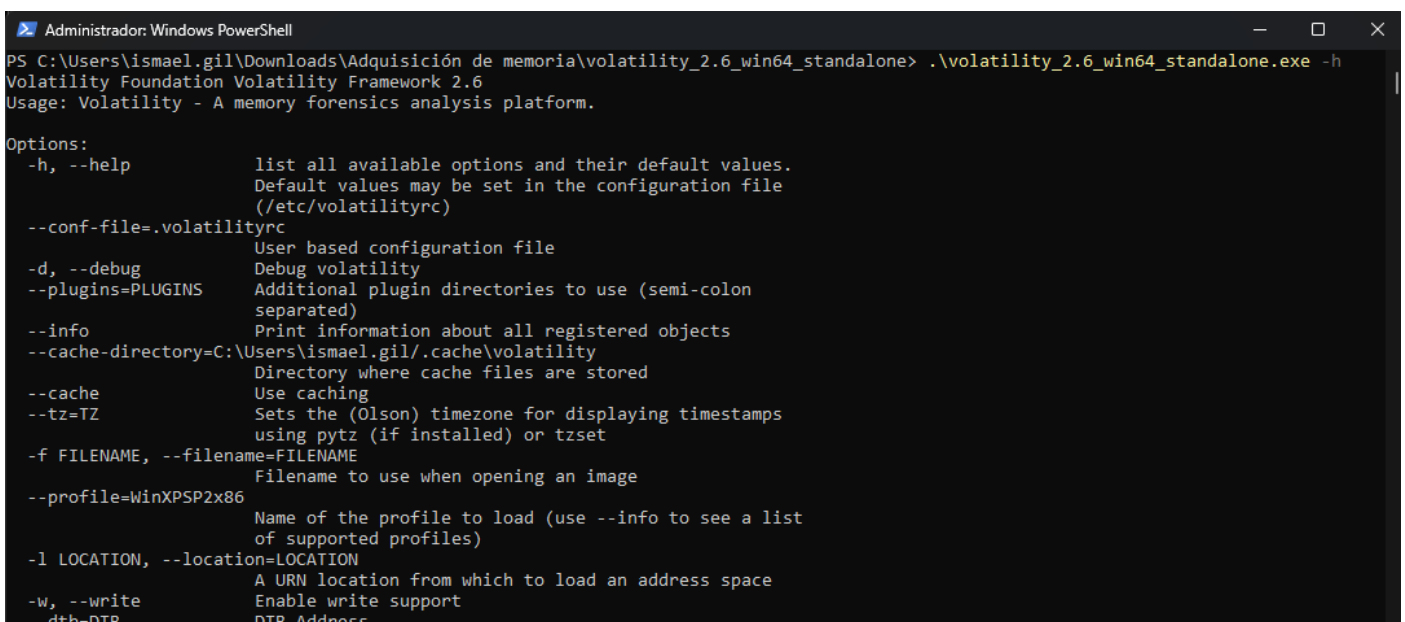
Tenemos que tener algo similar a:



Una vez tenemos el archivo descomprimido, abrimos un Powershell, con permisos de administrador y nos situamos en el directorio correspondiente, donde tenemos la aplicación



Si ejecutamos directamente el ejecutable por interfaz gráfica no vamos a poder ejecutar volatility, la manera de comprobar y lanzar volatility correctamente es la siguiente



Volatility V2 (Linux)

1. Instalar dependencias

Instalamos dependencias genericas

```
sudo apt-get update
```

```
sudo apt install -y build-essential git libdistorm3-dev yara  
libraw1394-11 libcapstone-dev capstone-tool tzdata
```

```
(ismael@hercules)-[~]  
$ sudo apt install -y build-essential git libdistorm3-dev yara libraw1394-11 libcapstone-dev capstone-tool tzdata  
[sudo] contraseña para ismael:  
build-essential ya está en su versión más reciente (12.12).  
Fijado build-essential como instalado manualmente.  
git ya está en su versión más reciente (1:2.51.0-1).  
Fijado git como instalado manualmente.  
libraw1394-11 ya está en su versión más reciente (2.1.2-2+b2).  
Fijado libraw1394-11 como instalado manualmente.  
libcapstone-dev ya está en su versión más reciente (5.0.6-1).  
Fijado libcapstone-dev como instalado manualmente.  
tzdata ya está en su versión más reciente (2025b-5).  
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.  
libbson-1.0-0t64 libjs-jquery-ui libjs-underscore libmongoc-1.0-0t64 libmongocrypt0 libxml2 python3-click-plugins python3-gpg python3-zombie-imp samba-ad-dc samba-ad-provision samba-dsdb-modules  
Utilice «sudo apt autoremove» para eliminarlos.  
  
Upgrading:  
  linux-image-amd64  
  
Installing:  
  capstone-tool libdistorm3-dev yara  
  
Installing dependencies:  
  libdistorm3-3 linux-image-6.16.8-kali-amd64  
  
Paquetes sugeridos:  
  linux-doc-6.16 debian-kernel-handbook  
  
Summary:  
  Upgrading: 1, Installing: 5, Removing: 0, Not Upgrading: 65  
  Download size: 116 MB  
  Space needed: 275 MB / 6.300 MB available
```

2. Instalar python2

```
sudo apt install -y python2 python2.7-dev libpython2-dev  
sudo curl https://bootstrap.pypa.io/pip/2.7/get-pip.py --output get-pip.py
```

```
sudo python2 get-pip.py  
sudo python2 -m pip install -U setuptools wheel
```

```
(ismael@hercules)-[~]  
$ sudo apt install -y python2 python2.7-dev libpython2-dev  
curl https://bootstrap.pypa.io/pip/2.7/get-pip.py --output get-pip.py  
$ sudo python2 get-pip.py  
$ sudo python2 -m pip install -U setuptools wheel  
python2 ya está en su versión más reciente (2.7.18-3).  
Fijado python2 como instalado manualmente.  
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.  
libbson-1.0-0t64 libjs-jquery-ui libjs-underscore libmongoc-1.0-0t64 libmongocrypt0 libxml2 python3-click-plugins python3-gpg python3-zombie-imp samba-ad-dc samba-ad-provision samba-dsdb-modules  
Utilice «sudo apt autoremove» para eliminarlos.  
  
Installing:  
  libpython2-dev python2.7-dev  
  
Installing dependencies:  
  libpython2.7 libpython2.7-dev  
  
Summary:  
  Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 65  
  Download size: 3.473 kB  
  Space needed: 16,2 MB / 5,950 MB available
```

3. Instalar volatility 2

```
python2 -m pip install -U distorm3 yara pycrypto pillow openpyxl ujson  
pytz ipython capstone
```

```
sudo python2 -m pip install yara
```

```
sudo ln -s /usr/local/lib/python2.7/dist-packages/usr/lib/libyara.so  
/usr/lib/libyara.so
```

```
python2 -m pip install -U
git+https://github.com/volatilityfoundation/volatility.git
```

4. Verificar instalación

```
(ismael@hercules)-[~]
$ vol.py -h
Volatility Foundation Volatility Framework 2.6.1
Usage: Volatility - A memory forensics analysis platform.

Options:
  -h, --help                list all available options and their default values.
                           Default values may be set in the configuration file
                           (/etc/volatilityrc)
  --conf-file=/home/ismael/.volatilityrc
                           User based configuration file
  -d, --debug               Debug volatility
  --plugins=PLUGINS         Additional plugin directories to use (colon separated)
  --info                    Print information about all registered objects
  --cache-directory=/home/ismael/.cache/volatility
                           Directory where cache files are stored
  --cache                   Use caching
  --tz=TZ                   Sets the (Olson) timezone for displaying timestamps
                           using pytz (if installed) or tzset
  -f FILENAME, --filename=FILENAME
                           Filename to use when opening an image
  --profile=WinXPSP2x86     Name of the profile to load (use --info to see a list
                           of supported profiles)
  -l LOCATION, --location=LOCATION
                           A URN location from which to load an address space
  -w, --write               Enable write support
  --dtb=DTB                 DTB Address
  --shift=SHIFT             Mac KASLR shift address
  --output=text             Output in this format (support is module specific, see
                           the Module Output Options below)
  --output-file=OUTPUT_FILE
                           Write output in this file
  -v, --verbose             Verbose information
  --physical_shift=PHYSICAL_SHIFT
                           Linux kernel physical shift address
  --virtual_shift=VIRTUAL_SHIFT
                           Linux kernel virtual shift address
  -g KDBG, --kdbg=KDBG      Specify a KDBG virtual address (Note: for 64-bit
                           Windows 8 and above this is the address of
                           KdCopyDataBlock)
  --force                   Force utilization of suspect profile
  --cookie=COOKIE           Specify the address of nt!ObHeaderCookie (valid for
                           Windows 10 only)
  -k KPCR, --kpcr=KPCR     Specify a specific KPCR address

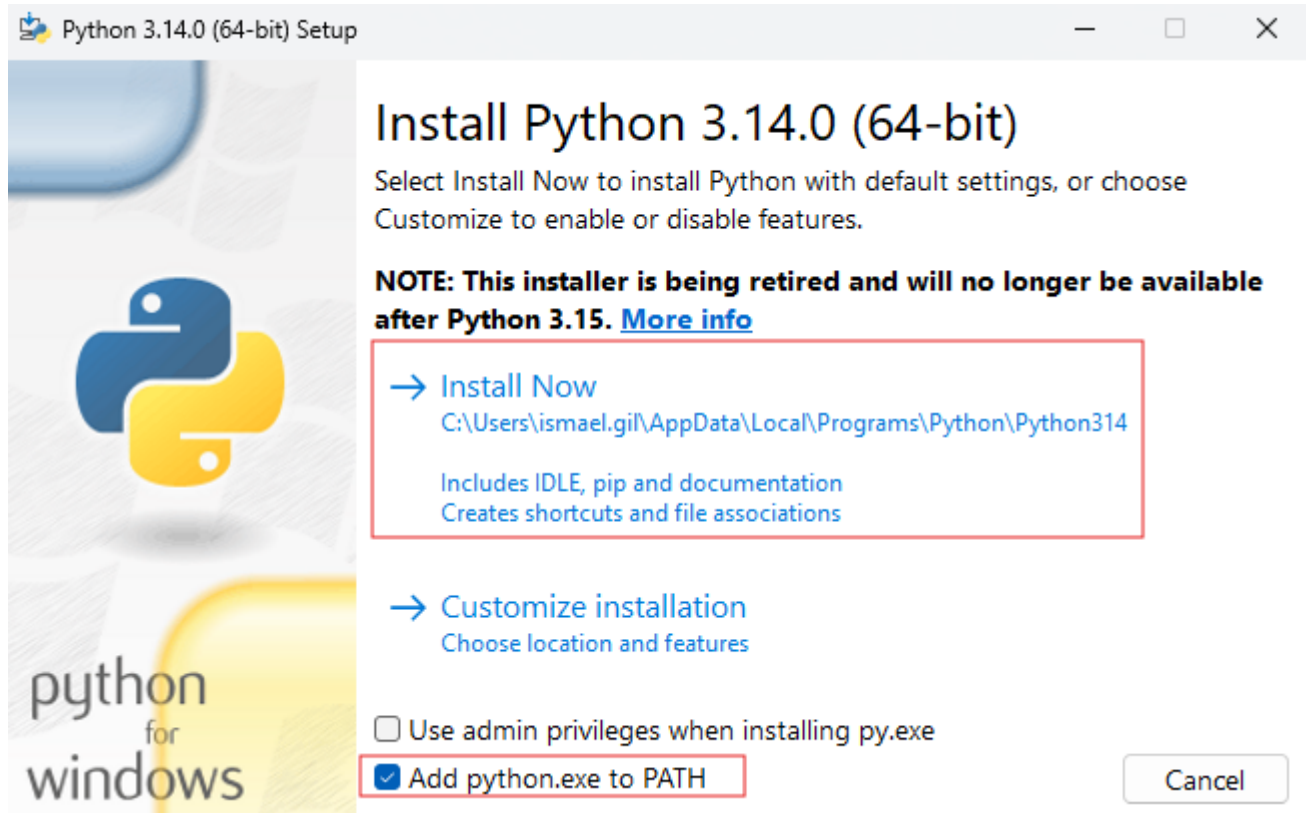
Supported Plugin Commands:
  amcache                   Print AmCache information
  apihooks                 Detect API hooks in process and kernel memory
  atoms                    Print session and window station atom tables
  atomscan                 Pool scanner for atom tables
  auditpol                 Prints out the Audit Policies from HKLM\SECURITY\Policy\PolAdtEv
```

Volatility V3 (Windows)

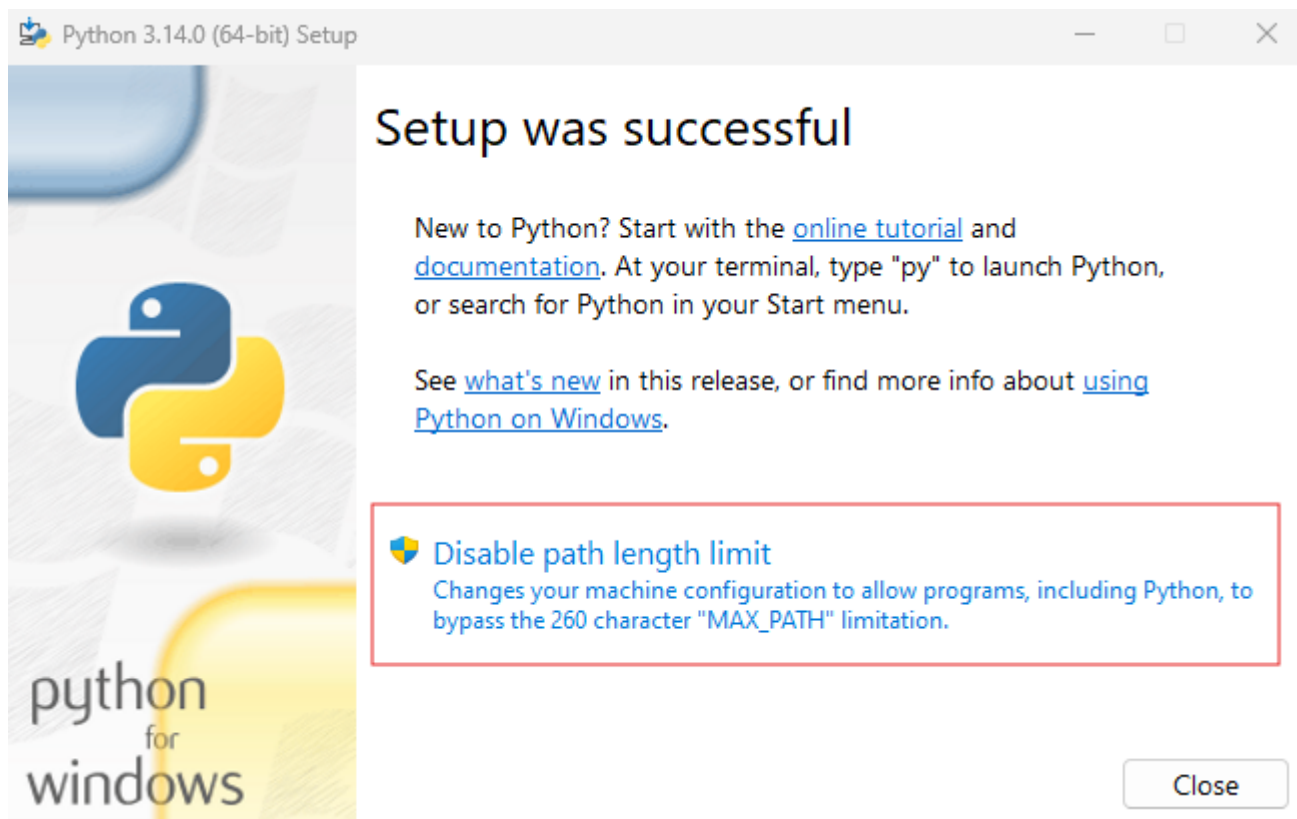
Para instalar volatility en su versión 3 en Windows debemos, en primer lugar, instalar python3 en nuestro equipo. Para ello iremos al repositorio oficial de Python y nos descargaremos la versión standalone en su última versión estable.



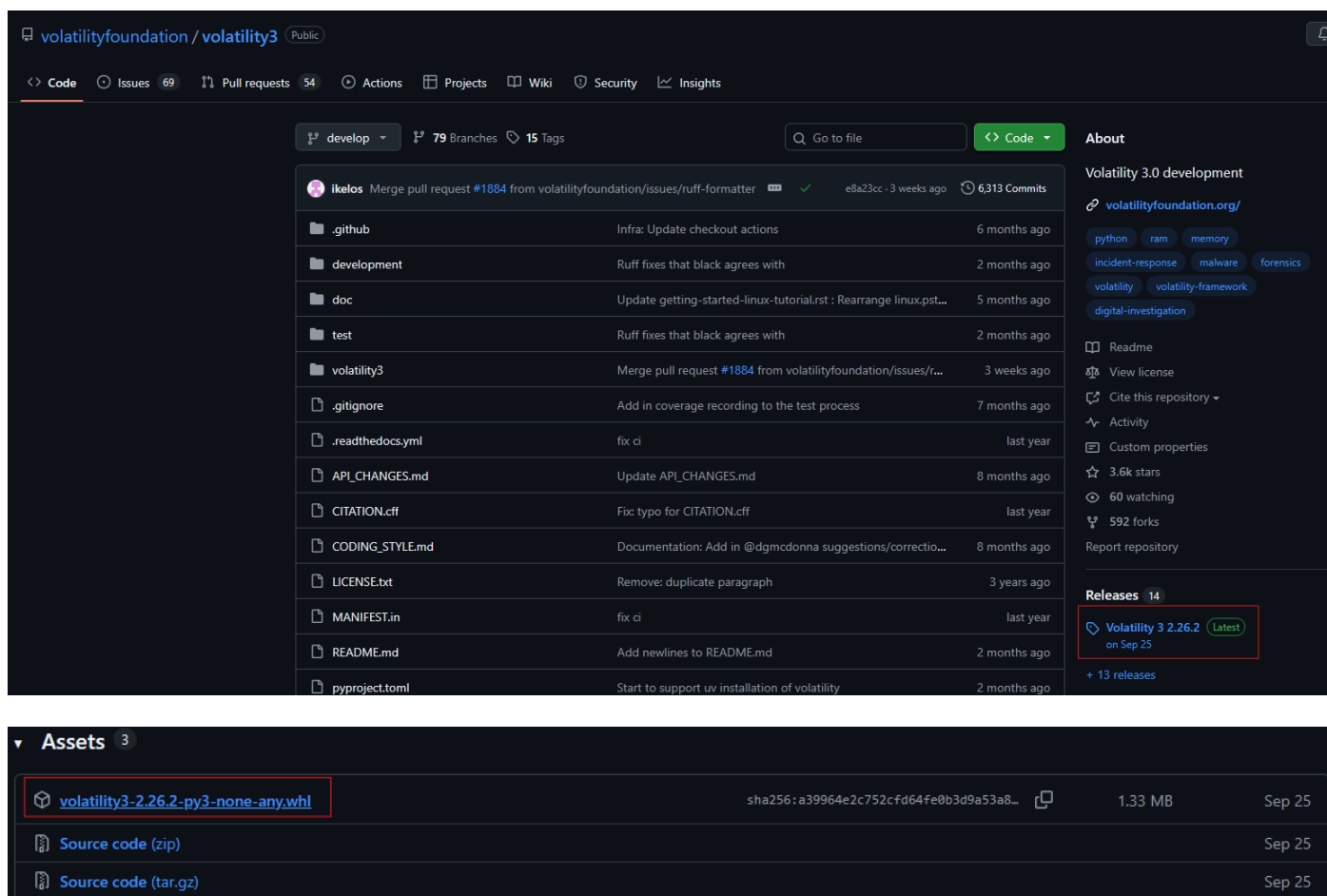
Ejecutamos el instalador, instalamos, añadiendo Python.exe a nuestro path para que podamos utilizarlo de manera más cómoda.



Importante por características de python3, deshabilitar la longitud mínima de caracteres del path.



Una vez tenemos python3 instalado, ya podemos descargar Volatility3, desde su repositorio oficial, <https://github.com/volatilityfoundation/volatility3/>



Una vez tenemos el archivo descargado, comprobamos que tenemos Python correctamente instalado y instalamos Volatility3

```
Administrador: Símbolo del sistema

C:\Users\ismael.gil\Downloads\Adquisición de memoria>python --version
Python 3.14.0

C:\Users\ismael.gil\Downloads\Adquisición de memoria>python -m pip --version
pip 25.2 from C:\Users\ismael.gil\AppData\Local\Programs\Python\Python314\Lib\site-packages\pip (python 3.14)

C:\Users\ismael.gil\Downloads\Adquisición de memoria>python -m pip install volatility3-2.26.2-py3-none-any.whl
Processing c:\users\ismael.gil\downloads\adquisición de memoria\volatility3-2.26.2-py3-none-any.whl
Collecting pefile>=2024.8.26 (from volatility3==2.26.2)
  Downloading pefile-2024.8.26-py3-none-any.whl.metadata (1.4 kB)
  Downloading pefile-2024.8.26-py3-none-any.whl (74 kB)
Installing collected packages: pefile, volatility3
  1/2 [volatility3] WARNING: The scripts vol.exe and volshell.exe are installed in 'C:\Users\ismael.gil\AppData\Local\Programs\Python\Python314\Scripts' which is not on PATH.
    Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed pefile-2024.8.26 volatility3-2.26.2

[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: C:\Users\ismael.gil\AppData\Local\Programs\Python\Python314\python.exe -m pip install --upgrade pip

C:\Users\ismael.gil\Downloads\Adquisición de memoria>
```

Lanzamos Volatility3 y vemos que esta correctamente operativo

```
Administrador: Símbolo del sistema

C:\Users\ismael.gil\Downloads\Adquisición de memoria>python -m pip show volatility3
Name: volatility3
Version: 2.26.2
Summary: Memory forensics framework
Home-page: https://github.com/volatilityfoundation/volatility3/
Author:
Author-email: Volatility Foundation <volatility@volatilityfoundation.org>
License: VSL
Location: C:\Users\ismael.gil\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requires: pefile
Required-by:

C:\Users\ismael.gil\Downloads\Adquisición de memoria>
```

Volatility V3 (Linux)

1. Instalar Volatility

```
git clone https://github.com/volatilityfoundation/volatility3.git
```

```
(ismael@hercules)-[~]
$ git clone https://github.com/volatilityfoundation/volatility3.git
Clonando en 'volatility3'...
remote: Enumerating objects: 49257, done.
remote: Counting objects: 100% (9239/9239), done.
remote: Compressing objects: 100% (1542/1542), done.
remote: Total 49257 (delta 8580), reused 7697 (delta 7697), pack-reused 40018 (from 2)
Recibiendo objetos: 100% (49257/49257), 9.92 MiB | 8.55 MiB/s, listo.
Resolviendo deltas: 100% (38183/38183), listo.

(ismael@hercules)-[~]
$ ls
Descargas  Documentos  Escritorio  get-pip.py  Imágenes  Música  Plantillas  Público  Videos  volatility3

(ismael@hercules)-[~]
$
```

2. Verificar instalación

Entramos en el directorio volatility3 y ejecutamos python3 vol.py y python3 vol.py -h

```
(ismael@hercules) ~/volatility3
$ python3 vol.py
Volatility 3 Framework 2.27.0
usage: vol.py [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]] [-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG] [-o OUTPUT_DIR] [-q] [-f FILE] [--write-config] [--save-config SAVE_CONFIG] [--clear-cache]
              [--cache-path CACHE_PATH] [--offline | -u URL] [--filters FILTERS] [--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER] [--single-location SINGLE_LOCATION] [--stackers [STACKERS ...]]
              [--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]]
              PLUGIN ...
vol.py: error: Please select a plugin to run (see 'vol.py --help' for options)

(ismael@hercules) ~/volatility3
$ python3 vol.py --help
usage: vol.py [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]] [-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG] [-o OUTPUT_DIR] [-q] [-f FILE] [--write-config] [--save-config SAVE_CONFIG] [--clear-cache]
              [--cache-path CACHE_PATH] [--offline | -u URL] [--filters FILTERS] [--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER] [--single-location SINGLE_LOCATION] [--stackers [STACKERS ...]]
              [--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]]
              PLUGIN ...

An open-source memory forensics framework

options:
  -h, --help            Show this help message and exit, for specific plugin options use 'vol.py <pluginname> --help'
  -c, --config CONFIG    Load the configuration from a json file
  --parallelism [{processes,threads,off}]
                        Enables parallelism (defaults to off if no argument given)
  -e, --extend EXTEND    Extend the configuration with a new (or changed) setting
  -p, --plugin-dirs PLUGIN_DIRS
                        Semi-colon separated list of paths to find plugins
  -s, --symbol-dirs SYMBOL_DIRS
                        Semi-colon separated list of paths to find symbols
  -v, --verbosity        Increase output verbosity
  -l, --log LOG          Log output to a file as well as the console
  -o, --output-dir OUTPUT_DIR
                        Directory in which to output any generated files
  -q, --quiet            Remove progress feedback
  -f, --file FILE        Shorthand for --single-location=file:// if single-location is not defined
  --write-config          Write configuration JSON file out to config.json
  --save-config SAVE_CONFIG
                        Save configuration JSON file to a file
  --clear-cache          Clears out all short-term cached items
  --cache-path CACHE_PATH
                        Change the default path (/home/ismael/.cache/volatility3) used to store the cache
  --offline              Do not search online for additional JSON files
  -u, --remote-isf-url URL
                        Search online for ISF json files
  --filters FILTERS      List of filters to apply to the output (in the form of [--]columnname,pattern[!])
  --hide-columns [HIDE_COLUMNS ...]
                        Case-insensitive space separated list of prefixes to determine which columns to hide in the output if provided
  -r, --renderer RENDERER
                        Determines how to render the output (quick, none, csv, pretty, json, jsonl, arrow, parquet)
  --single-location SINGLE_LOCATION
                        Specifies a base location on which to stack
  --stackers [STACKERS ...]
                        list of stackers
  --single-swap-locations [SINGLE_SWAP_LOCATIONS ...]
                        Specifies a list of swap layer URIs for use with single-location

Plugins:
For plugin specific options, run 'vol.py <plugin> --help'

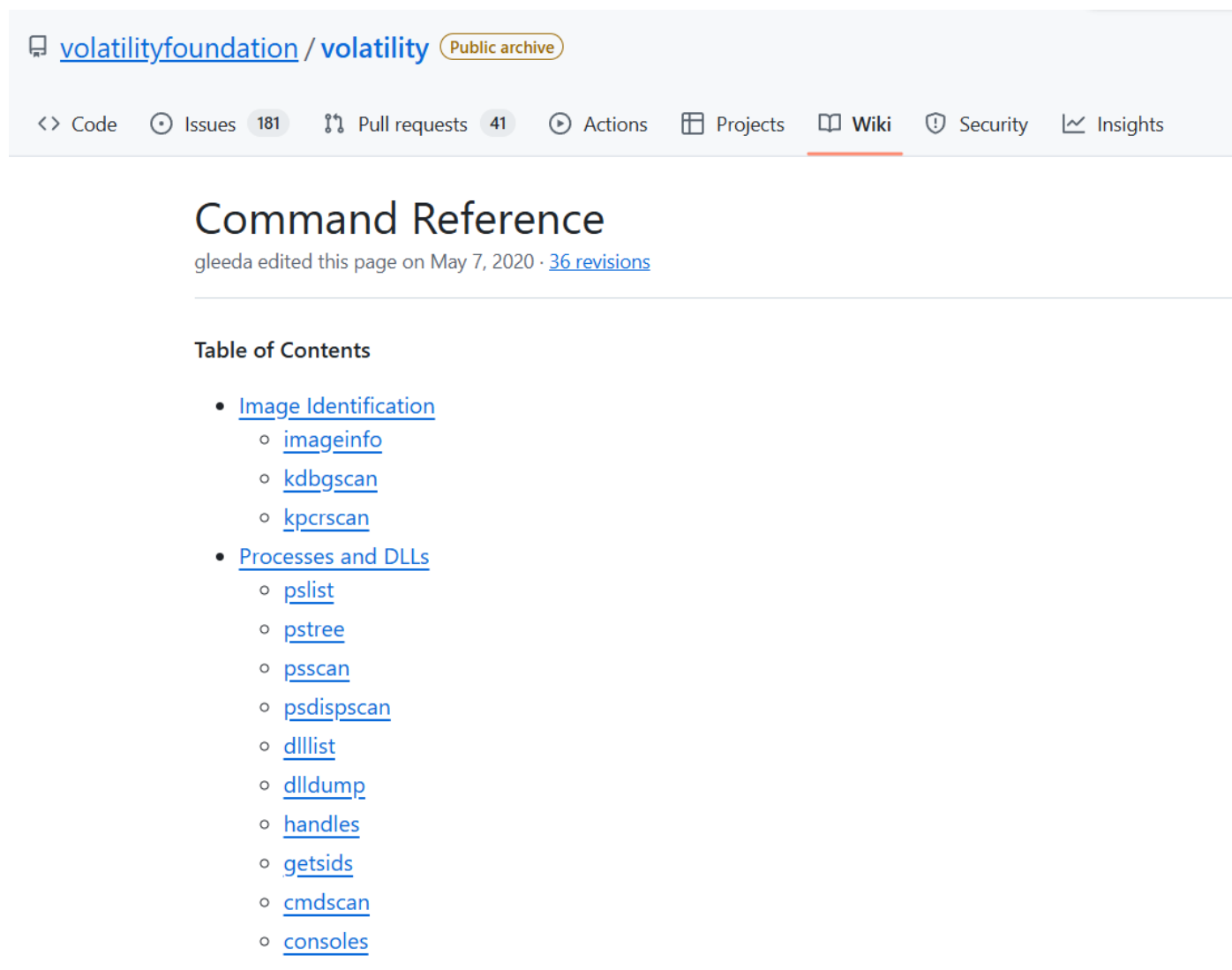
PLUGIN
  banners.Banners      Attempts to identify potential linux banners in an image
  configwriter.ConfigWriter
                        Writes the configuration and both prints and outputs configuration to the output directory
```

2.3.1.4 Funcionalidades de volatility 2

En la práctica vamos a utilizar Volatility v2 (a excepción que sea necesario analizar un volcado de Windows 11, **Volatility 2 llega hasta Windows 10**), ya que, pese a ser una versión que no tiene continuidad ya que esta basada en python2, es la que más plugins tiene, más información podremos encontrar y continúa siendo la más usada en la actualidad

Para encontrar las funciones de Volatility2 basta con ejecutar la aplicación con el argumento -h. No obstante, podéis consultar el “command reference” de la documentación oficial para que podáis tener todas las posibilidades que ofrece la herramienta junto con la información de cada una de ellas.

[Command reference](#)



The screenshot shows the GitHub interface for the [volatilityfoundation / volatility](#) repository, specifically the [Wiki](#) page titled **Command Reference**. The page is marked as a **Public archive**. The navigation bar includes links for Code, Issues (181), Pull requests (41), Actions, Projects, Wiki (selected), Security, and Insights. The page content includes the title **Command Reference**, a note that 'gleeda edited this page on May 7, 2020 · 36 revisions', and a **Table of Contents** section. The table of contents lists two main categories: **Image Identification** (with sub-items: [imageinfo](#), [kdbgscan](#), [kpcrscan](#)) and **Processes and DLLs** (with sub-items: [pslist](#), [pstree](#), [psscan](#), [psdispscan](#), [dlllist](#), [dlldump](#), [handles](#), [getsids](#), [cmdscan](#), [consoles](#)).

[volatilityfoundation](#) / [volatility](#) **Public archive**

<> Code Issues 181 Pull requests 41 Actions Projects **Wiki** Security Insights

Command Reference

gleeda edited this page on May 7, 2020 · [36 revisions](#)

Table of Contents

- [Image Identification](#)
 - [imageinfo](#)
 - [kdbgscan](#)
 - [kpcrscan](#)
- [Processes and DLLs](#)
 - [pslist](#)
 - [pstree](#)
 - [psscan](#)
 - [psdispscan](#)
 - [dlllist](#)
 - [dlldump](#)
 - [handles](#)
 - [getsids](#)
 - [cmdscan](#)
 - [consoles](#)

3 Listado de herramientas útiles

Volcado de memoria RAM

- WinPmem
- Magnet RAM Capture
- Mandiant Memoryze
- DumpIt
- FTKImager
- LiME

Análisis de memoria RAM

- [Volatility](#)
- Rakall

4 Referencias

- [WinPmem](#)
- [Magnet RAM Capture](#)
- [Mandiant Memoryze](#)
- [Avoid These Common Mistakes When Installing Volatility3 on Linux](#)
- [Introducción al análisis forense de memoria con Volatility 3](#)
- [VOLATILITY | Conviértete En Un Experto Forense Con Esta Herramienta](#)
- [Como Usar Volatility para Analizar Memoria RAM en Windows](#)
- [Volatility en Windows](#)
- [Volcado de memoria winpmem](#)
- [Instalar Volatility en Kali Linux](#)